AI-based Village Pond Planning System

Contour-map backend API for terrain-derived pond-site and catchment estimation

SUBMISSION STATUS
Backend implementation complete
KML and KMZ upload support

VALIDATION
4 automated tests passed
Sample contour map analyzed successfully

REPOSITORY

https://github.com/Galabavamsi/ai-village-pond-planning

VERIFIED API ACCESS

`http://127.0.0.1:18000/docs` (SSH-tunnel view from the developer PC)

`http://10.1.75.53:8000/docs` (private remote host, same network only)

Public reverse-proxy URL is not configured in this phase.

Prepared for Phase 2 submission

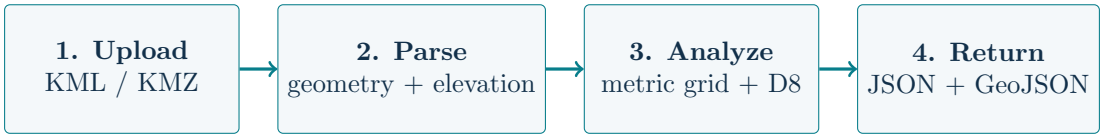
01 September 2026

1. Executive summary

This phase delivers a backend API that accepts a contour map in KML or KMZ format and derives terrain-based pond planning information. The service parses contour elevations, builds a local metric elevation grid, routes surface flow using D8 downhill routing, and returns ranked pond candidates with catchment area and GeoJSON geometries.

Generalization guarantee. No sample-specific coordinates, locations, or result values are embedded in the implementation. The supplied contour map is used only as a test input.

Phase 2 objectives



The API is structured as an independent analysis service so later phases can add rainfall, government-land availability, soil, satellite imagery, and a frontend map without replacing the core route.

2. Requirements traceability

Requirement	Implementation evidence	Status
Backend route	**POST /analyzeContour**; compatibility alias **/findCatchment**.	Complete
Upload contour map	Multipart upload field **file**; supports `.kml` and `.kmz`.	Complete
Analyze terrain	Contour parsing, interpolation to an elevation grid, and D8 flow routing.	Complete
Identify pond region	Ranks interior cells with high terrain-derived contributing flow and returns a pond footprint.	Complete
Estimate catchment	Upstream cells are merged into a GeoJSON Polygon or MultiPolygon with area metrics.	Complete
No hard-coding	Coordinates and recommendations are calculated from each uploaded input.	Complete
Documentation demo	README, this report, OpenAPI docs, automated tests, and sample-map results.	Complete

3. Catchment estimation methodology

1. **Parse KML/KMZ:** extract contour line geometry and elevation from placemark names or supported ExtendedData fields.
2. **Convert coordinates:** transform WGS84 longitude/latitude into local metre-based coordinates centered on the input extent.
3. **Interpolate terrain:** use the contour observations to build a regular elevation grid.

4. **Route flow:** route each cell to its lowest strictly lower neighbour in the eight-cell D8 neighbourhood.
5. **Accumulate area:** calculate the contributing cell area flowing into every cell.
6. **Rank candidates:** screen out the boundary, keep cells at or above the 85th percentile of accumulation, and prefer larger contributing area followed by lower elevation.
7. **Export geometry:** convert the selected point, pond region, and upstream catchment back into WGS84 GeoJSON.

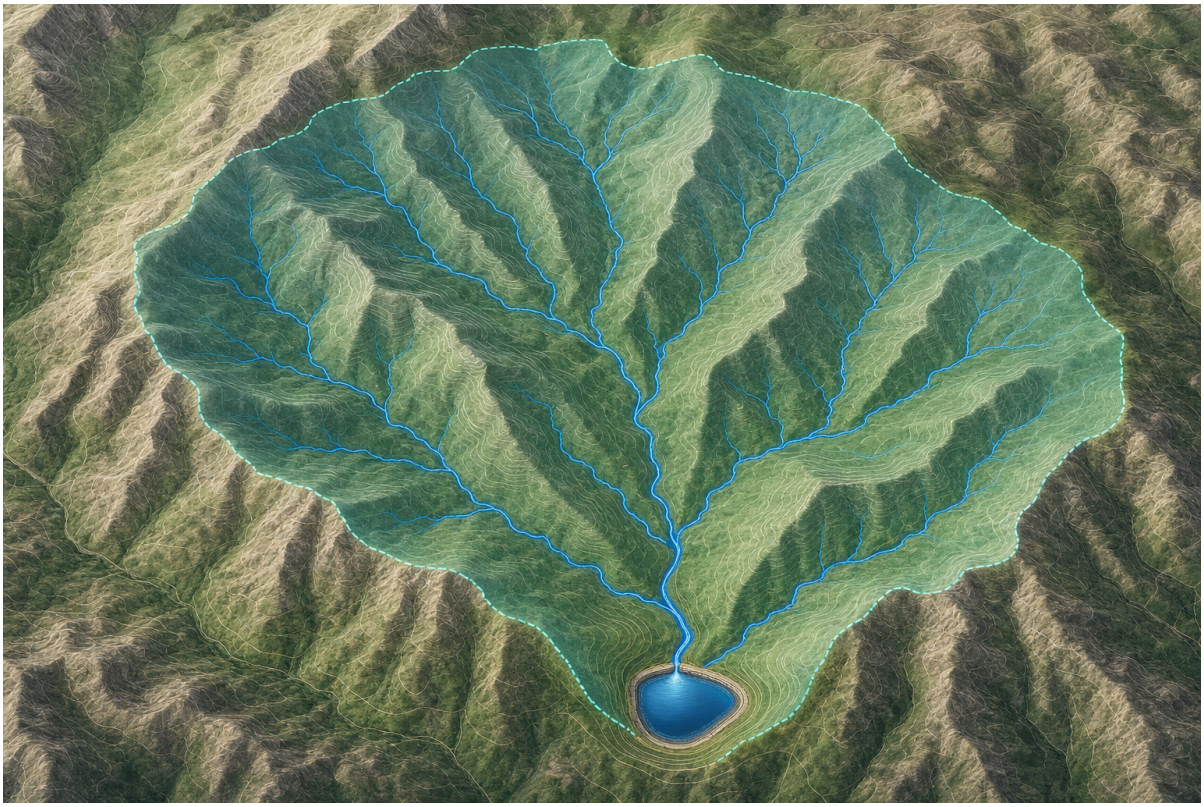


Figure 1: Conceptual catchment illustration generated for this report. The actual API derives boundaries from uploaded contours; this image is explanatory, not an analytical output.

Coordinate handling

Input coordinates are assumed to be standard WGS84 longitude/latitude as normally stored in KML. The analysis temporarily uses a local equirectangular approximation centered on the input extent so cell areas and flow distances are measured in metres. Returned geometries are converted back to WGS84 GeoJSON for direct map rendering.

Design rationale

A D8 flow model is transparent, reproducible, and extensible for this phase. It gives the frontend a meaningful catchment boundary while keeping later additions independent: rainfall can become a runoff-weighting layer, land ownership can become a constraint mask, and detailed hydrology can replace or refine the routing engine.

Core formulas

Let longitude λ, latitude ϕ, and the input-centre coordinates λ_0, ϕ_0 be in radians. Let $R = 6{,}371{,}000$ m.

$$x = R\cos(\phi_0)(\lambda - \lambda_0), \qquad y = R(\phi - \phi_0). \tag{1}$$

For grid spacings Δx and Δy, the area represented by one cell is

$$A_{\text{cell}} = |\Delta x\, \Delta y|\,. \tag{2}$$

For cell i, let $N_8(i)$ be its up-to-eight neighbouring cells and $z(i)$ its interpolated elevation. D8 routing selects the lowest strictly lower neighbour:

$$f(i) = \underset{n \in N_8(i),\ z(n) < z(i)}{\arg\min} z(n). \tag{3}$$

If no such neighbour exists, i is treated as a local sink.

The contributing area is accumulated from high cells toward lower cells:

$$A(i) = A_{\text{cell}} + \sum_{k:\, f(k) = i} A(k). \tag{4}$$

For a selected pond cell p, the catchment is the union of all cells that eventually route to p:

$$C(p) = \bigcup_{\text{cells } i \text{ with a flow path to } p} \text{cell}(i). \tag{5}$$

$$\text{catchment\_area}_{m^2} = \text{area}(C(p)), \tag{6}$$

$$\text{catchment\_area}_{ha} = \frac{\text{catchment\_area}_{m^2}}{10{,}000}. \tag{7}$$

The current phase uses a conservative screening estimate for pond storage:

$$d_{\text{pond}} = \text{clamp}(2.5\, CI,\ 1,\ 8)\ \text{m}, \tag{8}$$

$$V_{\text{storage}} = 0.75\, A_{\text{pond}}\, d_{\text{pond}}. \tag{9}$$

Here CI is the contour interval in metres and the factor 0.75 is a conservative effective-volume factor. In a later rainfall phase:

$$V_{\text{runoff}} = P\, A_{\text{catchment}}\, C_r, \tag{10}$$

where P is rainfall depth in metres and C_r is a locally justified runoff coefficient.

Algorithm summary

```
read uploaded KML or KMZ
extract contour geometries and elevation values
project coordinates to local metres around the input centre
interpolate observations onto a regular elevation grid
for every grid cell:
    choose the lowest strictly lower D8 neighbour
    otherwise mark the cell as a sink
sort cells from high elevation to low elevation
accumulate one cell area along each flow target
screen interior cells at or above the 85th accumulation percentile
rank by descending accumulation, then lower elevation
select spatially separated candidates
union all cells flowing to each candidate
return WGS84 GeoJSON and area/storage metrics
```

4. API documentation

Endpoint

```
POST /analyzeContour
Content-Type: multipart/form-data
Field: file=<contour-map.kml|contour-map.kmz>
```

Parameter	Type / default	Description
`grid_size`	integer / 100	Cells along the longer map dimension; range 40–220.
`max_candidates`	integer / 3	Number of ranked pond recommendations; range 1–5.

Success response

The JSON response contains analysis status, input contour statistics, terrain-grid metadata, and a list of recommendations. Each recommendation includes a WGS84 point, pond region, elevation, depth/storage estimate, and a catchment object with area in square metres/hectares and GeoJSON geometry.

```
{
  "analysis": {"status": "completed", "algorithm_version": "terrain-d8-v1"},
  "input": {"contour_features": 2710, "contour_interval_m": 1.0},
  "recommendations": [{
    "site_id": "pond-1",
    "location": {"type": "Point",
                 "coordinates": [81.2899372847, 21.2499875606]},
    "catchment": {"area_hectares": 16.0327,
                  "geometry": {"type": "MultiPolygon"}}
  }]
}
```

Additional routes and errors

GET /health returns service status. **GET /docs** opens interactive Swagger UI. **POST /findCatchment** is a compatibility alias. The API returns 415 for unsupported extensions, 413 for files over 75 MB, 400 for empty uploads, and 422 for invalid or unusable contour maps.

5. Demonstration using the supplied contour map

Test input: `contour-maps/contours_1m.kml` (approximately 6.7 MB). The endpoint was exercised through HTTP and returned status 200.

Observed metric	Result
Contour features	2,710
Elevation range	267–298 m
Contour interval	1 m
First recommended point	`81.2899372847, 21.2499875606`
First estimated catchment	16.0327 hectares
First storage estimate	4,233.98 m^3

Reproduce locally

```
python run.py
curl -X POST "http://127.0.0.1:8000/analyzeContour?grid_size=100&max_candidates=3" \
  -F "file=@contour-maps/contours_1m.kml"
```

The interactive alternative is `http://127.0.0.1:8000/docs`: expand **POST /analyzeContour**, select **Try it out**, upload the KML, and execute the request.

Figure 2: Conceptual rural rainwater-harvesting visual generated for this report. It provides context for the planning problem; the API evidence is shown in the following screenshots.

6. Interactive Swagger UI evidence

The screenshots document the complete demonstration flow: available routes, selected sample file, and successful HTTP 200 response.

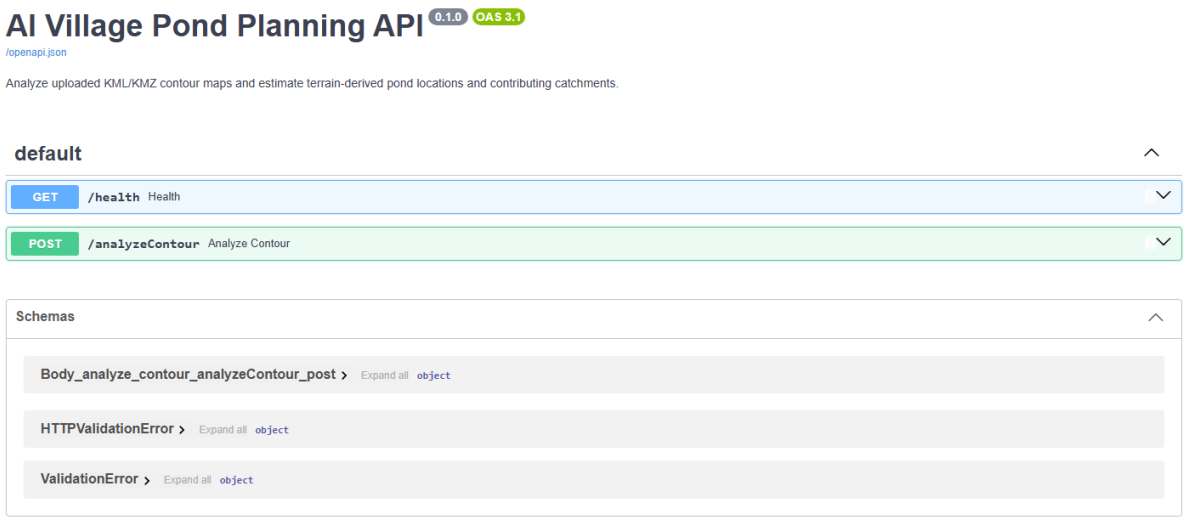


Figure 3: Swagger UI overview showing the health check and contour-analysis routes.

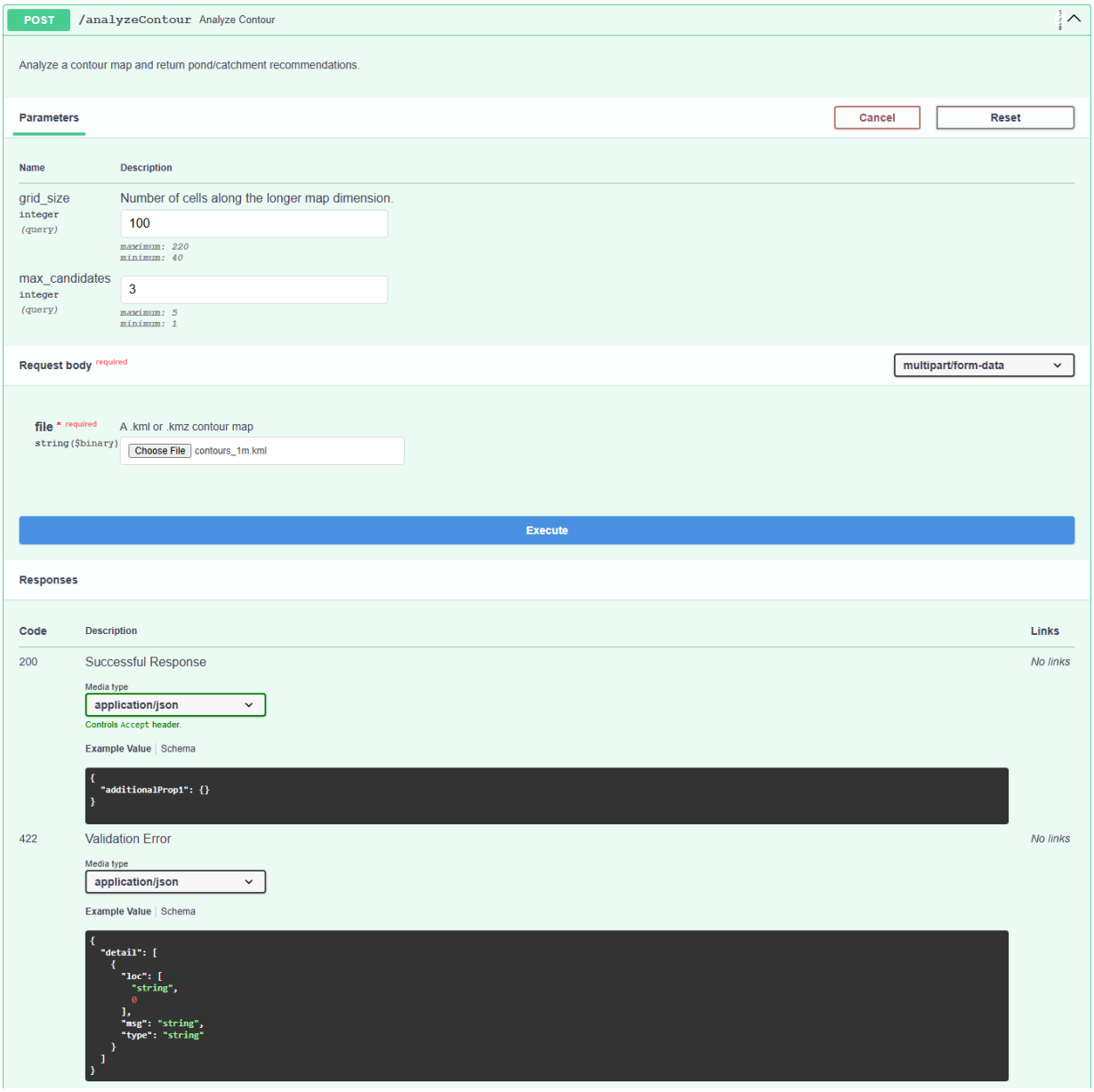


Figure 4: Expanded POST /analyzeContour form with the sample KML selected and Execute button ready.

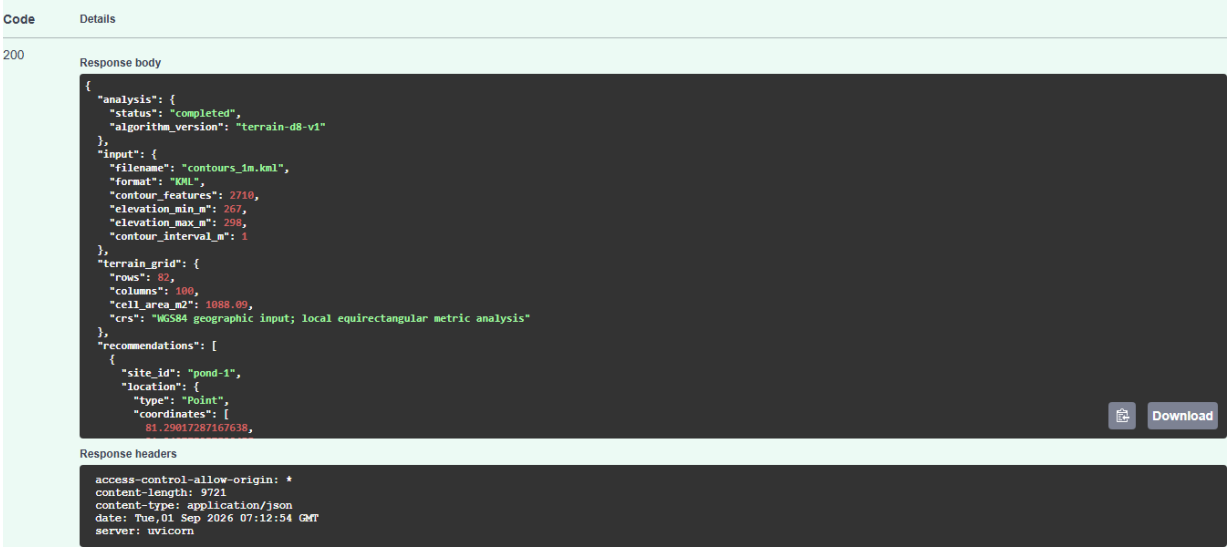


Figure 5: Successful HTTP 200 response showing completed analysis, contour statistics, terrain grid, and recommendation JSON.

The response can be consumed directly by a frontend map because pond and catchment geometries are returned as WGS84 GeoJSON.

7. Validation and deployment

Test	Purpose	Result
Health route	Confirms service responds.	Passed
Sample KML analysis	Confirms the full terrain pipeline.	Passed
Unsupported extension	Confirms upload validation.	Passed
Sample KMZ analysis	Confirms archive extraction.	Passed

Remote deployment

```
git clone https://github.com/Galabavamsi/ai-village-pond-planning.git
cd ai-village-pond-planning
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
tmux new -s pond-api
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The remote process is running in tmux session `pond-api`. The verified health response reports service status `ok` for `pond-planning-api`. The developer-PC tunnel exposes the same remote service at `http://127.0.0.1:18000`. A reverse proxy or secure tunnel is still required for a public evaluator URL. SSH credentials and tokens must remain outside the repository.

8. Limitations and future phases

The current result is a terrain-only planning estimate, not a final civil-engineering design. The reported storage estimate uses a conservative compact footprint and depth approximation derived from contour interval. It does not yet model rainfall intensity, runoff coefficient, infiltration, evaporation, land ownership, soil, structures, or field constraints.

Future layer	Planned extension
Rainfall	Apply rainfall and runoff coefficients to convert catchment area into seasonal volume scenarios.
Government land	Intersect candidate pond/catchment geometries with land-ownership boundaries.
Satellite data	Add land cover, drainage, vegetation, and water-body constraints.
Hydraulic design	Replace approximate footprint/depth with a stage-area-volume model.
Frontend map	Render returned GeoJSON on an interactive map and expose analysis parameters.

Conceptual AI-generated images are clearly identified; analytical claims are based on the implementation, tests, and captured API response.